Internship report

-Sanjeevni Kumari,sadiya ,Divya Arora

August 2025

1 Abstract

Diabetic retinopathy (DR) is a leading cause of preventable blindness worldwide, and early detection is essential for timely intervention. This project addresses automated classification of DR severity levels using the APTOS2019 retinal fundus image dataset, categorized into five severity grades. The dataset's significant class imbalance and subtle inter-class variations present notable challenges. To tackle these, we applied extensive preprocessing, class balancing with WeightedRandomSampler, and advanced data augmentation techniques. We experimented with multiple deep learning architectures, including EfficientNet, ResNet, EfficientNet–Swin Transformer hybrid, and EfficientNet–ResNet hybrid models. Each configuration was trained and evaluated, with the hybrid approaches designed to combine the convolutional feature extraction strength of CNNs with the global context learning capabilities of transformers or complementary CNN backbones. Among the experiments, the hybrid models achieved the best performance, with peak validation accuracy reaching 83

2 Literature review: Summary of existing work

2.1 Dataset & Task Framing

The APTOS 2019 Blindness Detection challenge provides a dataset of 3,662 training and 1,928 public test fundus photographs, with a private test set of approximately 13,000 images.[1, 2] The images were collected by Aravind Eye Hospital in India under diverse clinical conditions, resulting in variations in quality, focus, and lighting.[2, 3] Each image is labeled on the five-point International Clinical Diabetic Retinopathy (ICDR) severity scale: 0 (No DR), 1 (Mild), 2 (Moderate), 3 (Severe), and 4 (Proliferative DR).[4] The dataset is known for its significant class imbalance, with a large proportion of 'No DR' images and a scarcity of 'Severe DR' images.[2, 5] The official evaluation metric for the competition was the **quadratic weighted kappa (QWK)**, which measures agreement between predicted and human ratings and heavily penalizes large errors, making it ideal for this ordinal classification task.[6]

2.2 Methodology and Performance of Individual Research Papers

This section details the specific approaches and results from key academic papers and high-ranking Kaggle competition solutions.

2.2.1 Top Kaggle Competition Solutions

Guanshuo Xu (1st Place)

- **Methodology:** Xu's winning solution was an eight-model ensemble combining **2x Inception-ResNet-v2 (512x512), 2x Inception-v4 (512x512), 2x SE-ResNeXt50 (512x512), and 2x SE-ResNeXt101 (384x384)**.[7, 8] A critical strategy was augmenting the training data by combining the APTOS 2019 set with the entire 2015 Kaggle DR dataset.[7] In a notable departure from common practice, he used **"no special preprocessing, just plain resizing,"** arguing the

image quality was sufficient for deep networks.[7] The task was framed as a regression problem, using `nn.SmoothL1Loss()` as the loss function.[7] An architectural tweak involved replacing standard average pooling with **Generalized Mean Pooling (GeM)** to create more discriminative features.[7] Due to inconsistencies between local cross-validation (CV) and the public leaderboard (LB), he made a high-risk decision to rely solely on the public LB for validation.[7]

- **Performance:** Achieved the winning score on the private leaderboard, estimated to be a **QWK > 0.93**.[9]

Mykhailo Matviiv (2nd Place)

- **Methodology:** This solution was built on an ensemble of **EfficientNet models (B3, B4, and B5)**. The models were pre-trained on the 2015 Kaggle dataset before being fine-tuned on the 2019 data. The pipeline included extensive data augmentations from the Albumentations library, such as Blur, Flip, RandomBrightnessContrast, ShiftScaleRotate, and CLAHE. A key "secret ingredient" was the use of **pseudo-labeling**, where the model's predictions on the test set were used as new training labels for further fine-tuning, a process that provided a "huge boost" to the score.
- **Performance:** Achieved 2nd place on the private leaderboard.[9]

Qishen Ha (4th Place)

- **Methodology:** This team also relied heavily on the **EfficientNet family (B2-B7)**, stating that other architectures like SE-ResNeXt and DenseNet "performed poorly". Preprocessing involved a "Crop From Gray" function to remove black borders and a novel technique to normalize images based on boundary brightness. The final submission was an ensemble of six different EfficientNet models at varying resolutions (e.g., B7 at 224x224, B5 at 256x256). They used 5-fold CV, heavy Test-Time Augmentation (TTA), and pre-training on the 2015 dataset.
- **Performance:** The best single model (EfficientNet-B7) achieved a **private LB QWK of 0.929**. Their final ensemble submission achieved a **private LB QWK of 0.933**.

2.2.2 Peer-Reviewed Academic Papers

Tymchenko et al. (2020, *ICPRAM*)

- **Methodology:** This highly rigorous study proposed a **multi-task learning framework** with a shared CNN backbone (EfficientNet-B4/B5, SE-ResNeXt50) feeding into three separate heads: a classification head (using Focal Loss), a regression head (using MSE), and an ordinal regression head.[1] The final prediction was a linear combination of the outputs from all three heads.[1] The models were pre-trained on the 2015 EyePACs dataset and then fine-tuned on a combination of APTOS 2019, IDRiD, and MESSIDOR data.[1] They used extensive online augmentations from the Albumentations library to prevent overfitting.[1] Their final submission was a massive **ensemble of 20 models** (4 architectures x 5 folds) with heavy TTA.[1]
- **Performance:** The ensemble achieved a **private LB QWK of 0.925466**.[1] On their holdout set, the ensemble (without TTA) achieved a **QWK of 0.971** and an **accuracy of 92.9%** for the five-class problem.[1] For a simplified binary task, it achieved **99.1% sensitivity, 99.1% specificity, and 98.6% accuracy**.[1]

Yang et al. (2024, *PLOS ONE*)

- **Methodology:** This study developed a model to classify referable DR using a **Vision Transformer (ViT) with Masked Autoencoders (MAE)**. To overcome the data-hungry nature of transformers, they pre-trained the ViT on over 100,000 publicly available fundus images using the self-supervised MAE approach before fine-tuning for the classification task.
- **Performance:** The final model achieved an **accuracy of 93.42%**, an **AUC of 0.9853**, a sensitivity of 0.973, and a specificity of 0.9539.

Yi et al. (2021, referenced in Bodapati et al.)

- **Methodology:** This study proposed a new network called **RA-EfficientNet**, which integrates a residual attention (RA) mechanism into the EfficientNet architecture. The attention mechanism is designed to help the model focus on the most salient features in the fundus images relevant to DR grading.
- **Performance:** On the APTOS 2019 dataset, the RA-EfficientNet model achieved an **accuracy of 93.55%** and a **kappa score of 0.9193**.

Ahmed & Bhuiyan (2025, *arXiv*)

- **Methodology:** This paper presents a framework using **ResNet and EfficientNet variants** with a strong focus on addressing class imbalance. Their core contribution is an extensive, class-balanced data augmentation strategy that synthetically balances each DR severity class to ensure equal representation during training.
- **Performance:** For the five-class severity task, the model achieved a competitive **accuracy of 84.6%** and an AUC of 94.1%. For binary classification (DR vs. No DR), it achieved a state-of-the-art **accuracy of 98.9%** with an AUC of 99.4%.

Aftab & Akhtar (2025, *Journal of Software Engineering and Applications*)

- **Methodology:** This study fused three datasets (APTOS 2019, IDRiD, and Messidor-2) into a single corpus of 5,922 images. Preprocessing involved cropping, applying **CLAHE** to improve image quality, and using the **Synthetic Minority Over-sampling Technique (SMOTE)** to address class imbalance. An ensemble model was constructed by averaging the predictions of fine-tuned **EfficientNetB2, DenseNet121, and ResNet50** models.
- **Performance:** The final ensemble model achieved a **test accuracy of 96.96%** in grading all five DR classes.

Shorfuzzaman & Hossain (2021, *Symmetry*)

- **Methodology:** This study presented a non-CNN method based on extracting gray-level intensity and texture features from fundus images. These features were then classified using a decision tree-based ensemble learning technique.
- **Performance:** The approach yielded a **classification accuracy of 94.20%** with an F-measure of 93.51%.

Mondal et al. (2024, *Diagnostics*)

- **Methodology:** This study proposed an ensemble of modified **DenseNet101 and ResNeXt** models. The methodology included using CLAHE for preprocessing and GAN-based data augmentation to address the dataset's significant class imbalance.
- **Performance:** The model achieved a **five-class accuracy of 86.08%**.

2.3 Ongoing Challenges and Future Research Directions

Despite the high performance achieved, the literature consistently identifies several key challenges that define future research directions.

- **Interpretability and Explainable AI (XAI):** A major barrier to clinical adoption is the "black box" nature of deep learning models. Clinicians require insights into *why* a model makes a certain prediction to trust its output.[4]

- **Computational Efficiency and Deployment:** The large, computationally expensive ensembles that achieve top performance are often impractical for deployment in resource-constrained clinical settings, such as rural areas or on mobile devices.[10, 11]
- **Generalization and Domain Shift:** A critical question is whether models trained on the APTOS dataset, which is primarily from a rural Indian population, can generalize effectively to different patient populations, ethnicities, and images from different camera types.[2]
- **Label noise & ordinal ambiguity:** The APTOS labels are inherently ordinal and noisy, with potential "inconsistency between different doctors," especially for borderline cases.[1, 12] This creates an "unstable ground-truth" that can undermine the direct comparability of results across different papers and encourages model development that focuses on tuning to specific thresholds rather than true clinical generalization.[1, 12]

2.4 Future Research Directions

Based on the identified challenges, future research is likely to focus on several key areas:

- **Development of Interpretable Models:** Moving beyond simple accuracy metrics to create models that can generate clinically relevant explanations for their predictions, such as highlighting specific lesions on the fundus image that led to the diagnosis.
- **Lightweight and Efficient Architectures:** Continued research into model compression, knowledge distillation, and novel, efficient architectures (like advanced MobileNets or transformers) that can run on low-cost hardware is essential for real-world deployment.
- **Domain Generalization and Fairness:** Studies that explicitly test model performance on diverse, multi-ethnic datasets are needed to ensure that these diagnostic tools are equitable and robust when deployed globally.
- **Addressing Label Uncertainty:** Developing models that can handle or even quantify label uncertainty is a key frontier. This could involve probabilistic modeling or training frameworks that are robust to the inherent noise in subjective medical grading.

3 final problem statement

1. Background Diabetic Retinopathy (DR) is a progressive eye disease caused by damage to the retina's blood vessels due to prolonged high blood sugar levels. It is a major cause of preventable blindness worldwide, particularly affecting individuals with diabetes. DR progresses through multiple stages — from No DR to Proliferative DR — each requiring accurate and timely diagnosis to prevent irreversible vision loss.

2. Current Challenges Traditional diagnosis relies on manual examination of retinal fundus images by ophthalmologists. While effective, this process is: Time-consuming for large-scale screening programs Resource-intensive, requiring trained specialists and clinical facilities Subjective, leading to variations in interpretation between experts Limited in scalability, especially in rural or under-resourced regions

3. Motivation Advancements in artificial intelligence (AI) and deep learning offer the potential to develop automated, accurate, and scalable systems for DR detection. Such systems can assist healthcare professionals in early diagnosis, reduce the workload on ophthalmologists, and make screening more accessible.

4. Problem Definition There is a need for a robust and reliable automated system capable of: Detecting DR from retinal fundus images Classifying its severity across multiple stages Handling variations in image quality and class distribution Providing interpretable results that can support clinical decision-making

5. Scope of the Problem This project aims to: Utilize publicly available retinal image datasets for model development and evaluation Implement effective preprocessing and augmentation techniques to improve robustness Evaluate performance across accuracy, precision, recall, F1-score, and ROC-AUC Integrate explainability tools to enhance transparency and trust in the predictions

4 DataSet Description

The dataset used in this project is the APTOS 2019 Blindness Detection dataset, which contains retinal fundus photographs collected from patients during routine diabetes screening. Each image is labeled with one of five classes representing the severity of Diabetic Retinopathy (DR).

4.1 Classes

Label	Severity Level	Description
0	No DR	No signs of diabetic retinopathy
1	Mild	Microaneurysms only, earliest detectable stage.
2	Moderate	Both microaneurysms and hemorrhages present.
3	Severe	Large number of hemorrhages and blood vessel abnormalities.
4	Proliferative DR	Advanced stage, new abnormal blood vessels form, high risk of vision loss.

4.2 Dataset Structure

The dataset is split into training and validation subsets. A total of 3,296 images were used, split into:

Training set: 2,930 images

Validation set: 366 images

Files Directories:

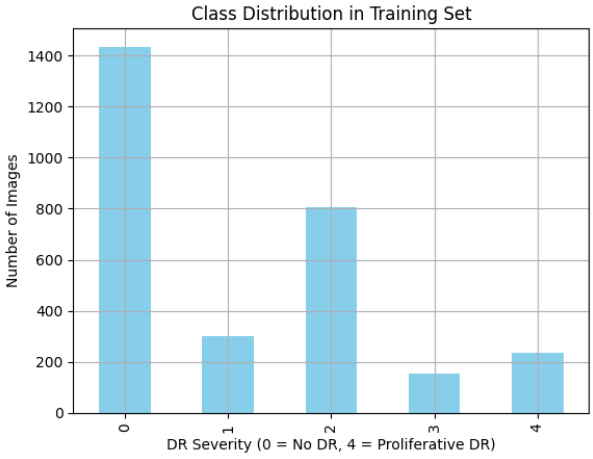
train.csv — Contains two columns: id_code (image filename without extension) and diagnosis (label 0–4).

valid.csv — Same structure as train.csv but used for validation.

train_images/ — JPEG/PNG images for training.

val_images/ — JPEG/PNG images for validation.

4.3 Class Distribution



4.4 Preprocessing Data Augmentation

To ensure compatibility with deep learning architectures and improve generalization, the following preprocessing steps were applied:

Resizing: All images resized to 224×224 pixels.

Normalization: Applied ImageNet mean [0.485, 0.456, 0.406] and standard deviation [0.229, 0.224, 0.225].

Color conversion: All images converted to RGB.

For the training set, augmentation techniques included:

Random horizontal flip (p = 0.5)

Random rotation (±15°)

Brightness and contrast adjustment (±20

Random resized crop (scale range: 0.8–1.0)

For the validation set, only resizing, tensor conversion, and normalization were applied to maintain consistency during evaluation.

```
import torchvision.transforms as transforms

# ImageNet mean & std
mean = [0.485, 0.456, 0.406]
std = [0.229, 0.224, 0.225]

# Augmentation for training set
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
    transforms.ToTensor(),
    transforms.Normalize(mean, std)
])

# Validation transform (no augmentation)
val_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean, std)
])
```

4.5 Custom Dataset Loader

A custom PyTorch Dataset class, DRDataset, was implemented to:

Load images and corresponding labels from the dataset directories using the PIL library.

Apply the defined transformation pipeline (augmentation for training, standard preprocessing for validation).

Return image tensors and integer labels for model input.

This method ensured on-the-fly loading and augmentation without loading the entire dataset into memory.

```
from PIL import Image
from torch.utils.data import Dataset

class DRDataset(Dataset):
    def __init__(self, dataframe, img_dir, transform=None):
        self.dataframe = dataframe
        self.img_dir = img_dir
        self.transform = transform

    def __len__(self):
```

```
        return len(self.dataframe)

    def __getitem__(self, idx):
        img_name = str(self.dataframe.iloc[idx]['id_code']) + ".png"
        label = int(self.dataframe.iloc[idx]['diagnosis'])
        img_path = os.path.join(self.img_dir, img_name)

        image = Image.open(img_path).convert('RGB')

        if self.transform:
            image = self.transform(image)

        return image, label
```

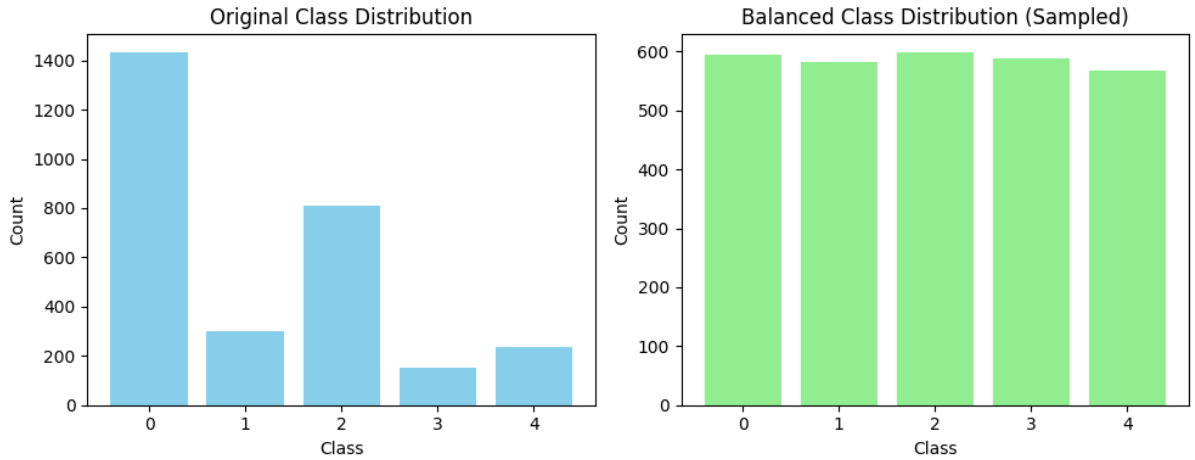
4.6 Handling Class Imbalance

Due to severe class imbalance, a Weighted Random Sampler was implemented: Class counts were calculated from the training set. Class weights were assigned inversely proportional to their frequency. Sample weights were mapped to each training example. The WeightedRandomSampler ensured that each mini-batch contained a balanced class distribution.

```
import torch
from torch.utils.data import DataLoader, WeightedRandomSampler
import numpy as np

# Compute class counts
class_counts = train_df['diagnosis'].value_counts().sort_index()
class_weights = 1. / class_counts
sample_weights = train_df['diagnosis'].map(class_weights).values

sampler = WeightedRandomSampler(weights=torch.DoubleTensor(sample_weights),
                                num_samples=len(sample_weights),
                                replacement=True)
```



5 Experiments

5.1 Experiment 1: Hybrid EfficientNet-B0 + ResNet18

5.1.1 Objective

To leverage the complementary feature extraction capabilities of EfficientNet-B0 and ResNet18 for diabetic retinopathy classification.

5.1.2 Model Architecture

EfficientNet-B0: Pretrained on ImageNet, producing 1,280-dimensional features.

ResNet18: Pretrained on ImageNet, producing 512-dimensional features.

Fusion Layer: Concatenation of features (total 1,792 dimensions).

Classifier:

Linear layer (1,792 $\rightarrow$ 512), ReLU activation, Dropout(0.4) Linear layer (512 $\rightarrow$ 5 classes)

5.1.3 Training Configuration

Loss Function: CrossEntropyLoss

Optimizer: Adam (learning rate = 1e-4)

Batch Size: 32

Epochs: 4

Device: CUDA-enabled GPU **Class Balance:** Not applied — the dataset was used in its original imbalanced form

5.1.4 Results

Validation Accuracy: 81% after 4 epochs.

Accuracy: 0.8142076502732241

Class	Precision	Recall	F1-Score	Support
No DR	0.99	0.99	0.99	172
Mild	0.62	0.70	0.66	40
Moderate	0.73	0.77	0.75	104
Severe	0.31	0.18	0.23	22
Proliferative DR	0.56	0.54	0.55	28
Accuracy			0.81	366
Macro Avg	0.64	0.64	0.64	366
Weighted Avg	0.80	0.81	0.81	366

Table 1: Classification report showing precision, recall, F1-score, and support.

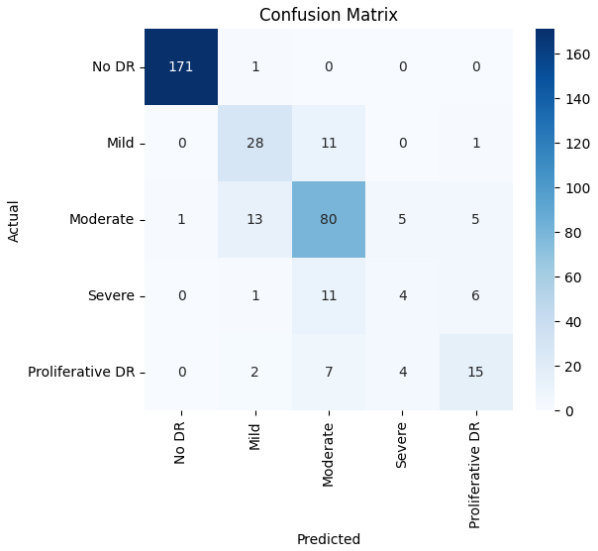


Figure 1: confusion matrix

5.2 Experiment 2:Hybrid EfficientNet-B0 + ResNet18 with Class Balancing

5.2.1 Objective

To improve minority class performance by addressing the dataset's inherent class imbalance during training.

5.2.2 Model Architecture

Same as Experiment 1:

EfficientNet-B0 (1,280 features) + ResNet18 (512 features)

Features concatenated → fully connected layers → 5-class output

5.2.3 Balancing Strategy:

Implemented WeightedRandomSampler in PyTorch.

Sampling probability inversely proportional to class frequency.

Ensured each batch contained a more uniform distribution of all classes.

5.2.4 Training Configuration

Loss Function: CrossEntropyLoss

Optimizer: Adam (learning rate = 1e-4)

Batch Size: 32

Epochs: 10

Device: CUDA-enabled GPU

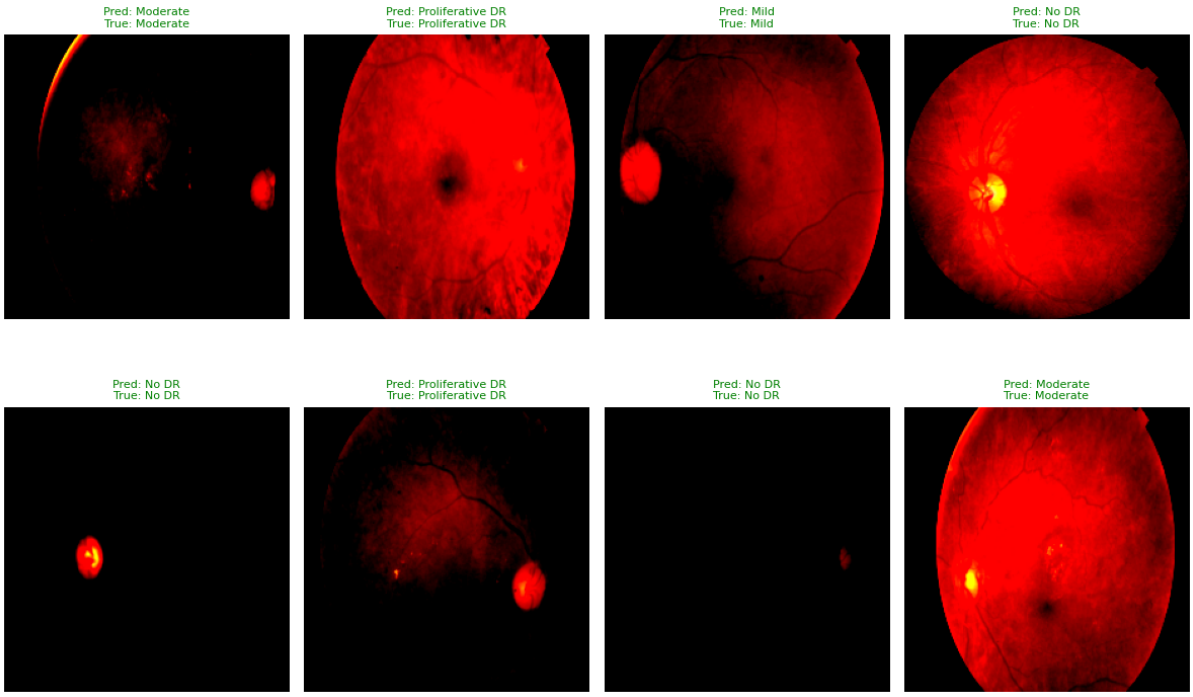


Figure 2: confusion matrix

5.2.5 Results

Validation Accuracy: 82.24% after 10 epochs.

Class balancing led to a noticeable improvement (+1.24% over Experiment 1) in validation accuracy. The model likely learned better representations for minority classes, but performance gains were modest—indicating that balancing alone is not enough and may need to be combined with stronger augmentation or advanced loss functions.

5.3 Experiment 3:Weighted Cross-Entropy Loss + Data Augmentation + Label Smoothing (Without Class Balancing)

5.3.1 Objective

To improve accuracy by modifying the loss function and enhancing training data diversity, without explicitly balancing the dataset via sampling.

5.3.2 Model architecture

Same as Experiment 1:

Hybrid EfficientNet-B0 + ResNet18 model.

Feature concatenation → Fully connected layers → 5-class output.

5.3.3 Modifications Over Baseline

1.Weighted Cross-Entropy Loss

Class weights computed inversely proportional to their frequency in the training set.

Encourages better handling of underrepresented classes without using a sampler

2.Data Augmentation Applied during training:

Random horizontal flip

Random rotation (15°)

Random resized crop (scale 0.8–1.0)

Color jitter (brightness contrast adjustments)

Normalized using ImageNet mean and std values.

Label Smoothing Value = 0.1

Helps regularize predictions, avoiding overconfidence and improving generalization.

Extended Training Epochs From 4 → 30 to allow better convergence

5.3.4 Training Configuration

Loss Function: Weighted CrossEntropyLoss with label smoothing = 0.1

Optimizer: Adam (lr = 1e-4)

Batch Size: 32

Epochs: Initially 4, later increased to 30

5.3.5 Results

After first modifications: 82.79% validation accuracy

After increasing epochs to 30: 83.06% validation accuracy

5.4 Experiment 4: Mixup SMOTE Augmentation

5.4.1 Objective

To further improve model generalization and address class imbalance by combining data-level synthetic augmentation techniques.

5.4.2 Modifications Over Previous Experiments:

1. MixUp Augmentation

MixUp is a data augmentation technique that randomly combines two training samples (both images and their labels) in a weighted manner.

Formulation:

$$\tilde{x} = \lambda x_i + (1 - \lambda) x_j$$
$$\tilde{y} = \lambda y_i + (1 - \lambda) y_j$$

where $\lambda \sim Beta(\alpha, \alpha)$ and $\alpha = 0.4$.

Description:

- Two samples (x_i, y_i) and (x_j, y_j) are randomly chosen from the dataset.

- They are linearly combined using a mixing parameter λ drawn from the Beta distribution.
- This reduces overfitting and improves robustness to noisy labels.

2. SMOTE (Synthetic Minority Oversampling Technique) Applied on the training set before batching.

Generates synthetic feature-space examples for minority classes by interpolating between existing samples.

Helps address class imbalance without duplicating the same samples.

3. Loss Function CrossEntropyLoss with soft targets from Mixup.

No label smoothing in this experiment.

4.Data Augmentation Standard augmentation pipeline (random crop, horizontal flip, color jitter, normalization).

5.4.3 Training Configuration

Loss Function: CrossEntropyLoss with Mixup soft labels

Optimizer: Adam (lr = 1e-4)

Batch Size: 32

Epochs: 30

Sampling: SMOTE applied before training, so batches were drawn from a balanced dataset

5.4.4 Results

Validation Accuracy: 84

5.4.5 Observation

This approach achieved the highest accuracy among all experiments so far.

Mixup improved generalization by blending image-label pairs.

SMOTE helped the model see more balanced data distributions, improving minority-class recall.

Compared to class-weighting methods, SMOTE produced more stable convergence across epochs.

5.5 Experiment 5: Hybrid EfficientNet-B0 + ResNet18 with Mixup Augmentation

5.5.1 Objective

To improve model generalization and robustness by combining a hybrid CNN backbone with **Mixup augmentation**, aiming to reach higher validation accuracy while addressing overfitting.

5.5.2 Model Architecture

Hybrid EfficientNet-B0 + ResNet18:

- **EfficientNet-B0:** Pretrained on ImageNet, outputs 1280-dimensional features.
- **ResNet18:** Pretrained on ImageNet, outputs 512-dimensional features.
- **Fusion Layer:** Concatenation (1792D vector) → Linear(1792 → 512) → ReLU → Dropout(0.5) → Linear(512 → 5).

5.5.3 Techniques Applied

1. **Data Augmentation:** Resize (224×224), horizontal/vertical flips, random rotation, brightness/contrast adjustment, CLAHE (contrast-limited adaptive histogram equalization), normalization.
2. **Mixup Augmentation:** Images and labels are linearly interpolated:

$$\tilde{x} = \lambda x_i + (1-\lambda)x_j, \quad \tilde{y} = \lambda y_i + (1-\lambda)y_j, \quad \lambda \sim Beta(\alpha, \alpha),\ \alpha = 1.0$$

   This smooths decision boundaries and improves generalization.
3. **Loss Function:** CrossEntropyLoss with Mixup criterion.
4. **Optimizer:** Adam (learning rate = $1e^{-4}$).

5.5.4 Training Configuration

- **Batch Size:** 32
- **Epochs:** 20
- **Device:** CUDA-enabled GPU
- **Class Balance:** None (Mixup served as implicit regularization)

5.5.5 Results

- Validation Accuracy: **86%** after 20 epochs.
- Mixup significantly improved robustness, reducing overfitting compared to earlier experiments.
- Minority-class recall improved moderately due to smoother decision boundaries.

```
--- Validation Metrics for Epoch 20 ---
                  precision    recall  f1-score   support

          No DR       0.98      0.99      0.99       172
           Mild       0.72      0.65      0.68        40
       Moderate       0.76      0.90      0.83       104
         Severe       0.64      0.41      0.50        22
Proliferative DR      0.70      0.50      0.58        28

       accuracy                           0.86       366
      macro avg       0.76      0.69      0.72       366
   weighted avg       0.85      0.86      0.85       366
```

5.5.6 Conclusion

The **Hybrid EfficientNet-B0 + ResNet18 with Mixup augmentation** achieved the **best performance (86% accuracy)** among all experiments. Mixup regularization improved generalization by encouraging the model to learn smoother class boundaries, while extensive augmentations further enhanced robustness. Future work could explore combining Mixup with advanced balancing strategies (e.g., SMOTE + Mixup) or integrating transformer-based backbones for further improvements.

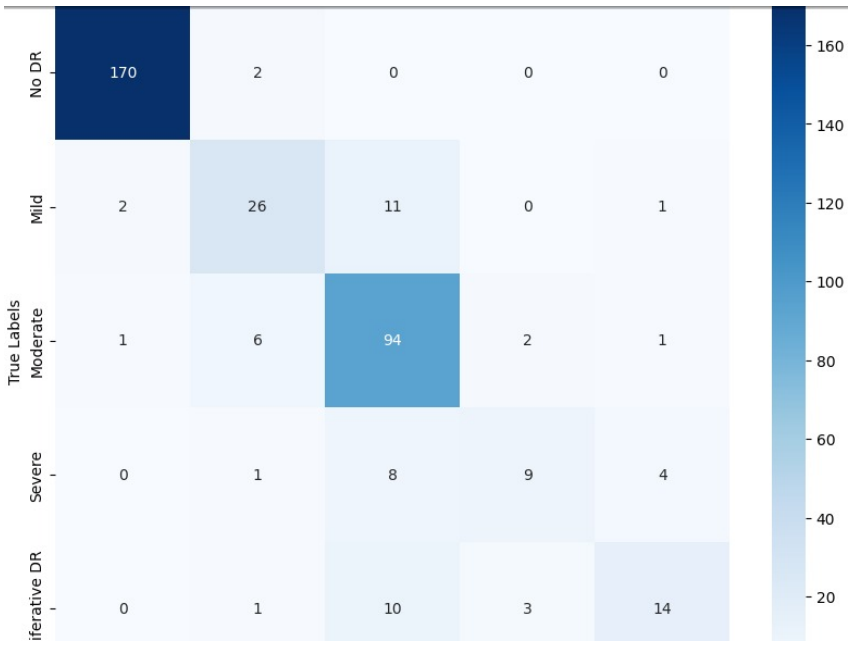


Figure 3: Confusion matrix of Hybrid EfficientNet-B0 + ResNet18 with Mixup augmentation.

5.6 Experiment 6: Hybrid Model of EffNet and Swin Transformer

5.6.1 Objective

A hybrid model was developed by integrating EfficientNet-B4 and Swin Transformer to leverage the strengths of both CNN-based and Transformer-based feature extraction.

5.6.2 Model Components:

`1. EfficientNet-B4 Backbone (timm.create_model('efficientnet_b4', pretrained=True, features_only=Tru`

Extracts hierarchical convolutional features from input images.

Output: Final convolutional feature map.

3. 1×1 Convolution (Conv2d) Layer:

Reduces the EfficientNet output channels to 3, allowing compatibility with the Swin Transformer input format.

`2. Swin Transformer Base (timm.create_model('swin_base_patch4_window7_224', pretrained=True)):`

Processes spatially structured features to capture long-range dependencies.

3. Fully Connected Classifier:

LayerNorm on the Swin output features. Linear layer projecting to the 5 output classes of diabetic retinopathy.

Forward Pass: Input image → EfficientNet feature extraction → 1×1 Conv layer → Upsampling to 224×224 → Swin Transformer → Fully connected classifier → Output logits.

Parameter	Value
Optimizer	Adam
Loss Function	CrossEntropyLoss (with class weights)
Learning Rate	0.0001 (replace if different)
Batch Size	32
Epochs	(enter value)
Data Augmentation	Horizontal flip, rotation, brightness/contrast adjustment, random resized crop
Sampler	WeightedRandomSampler
Validation	No augmentation, fixed normalization

Table 2: Training parameters and settings.

5.6.3 Training Configuration

5.6.4 Result

Validation Accuracy: 82%

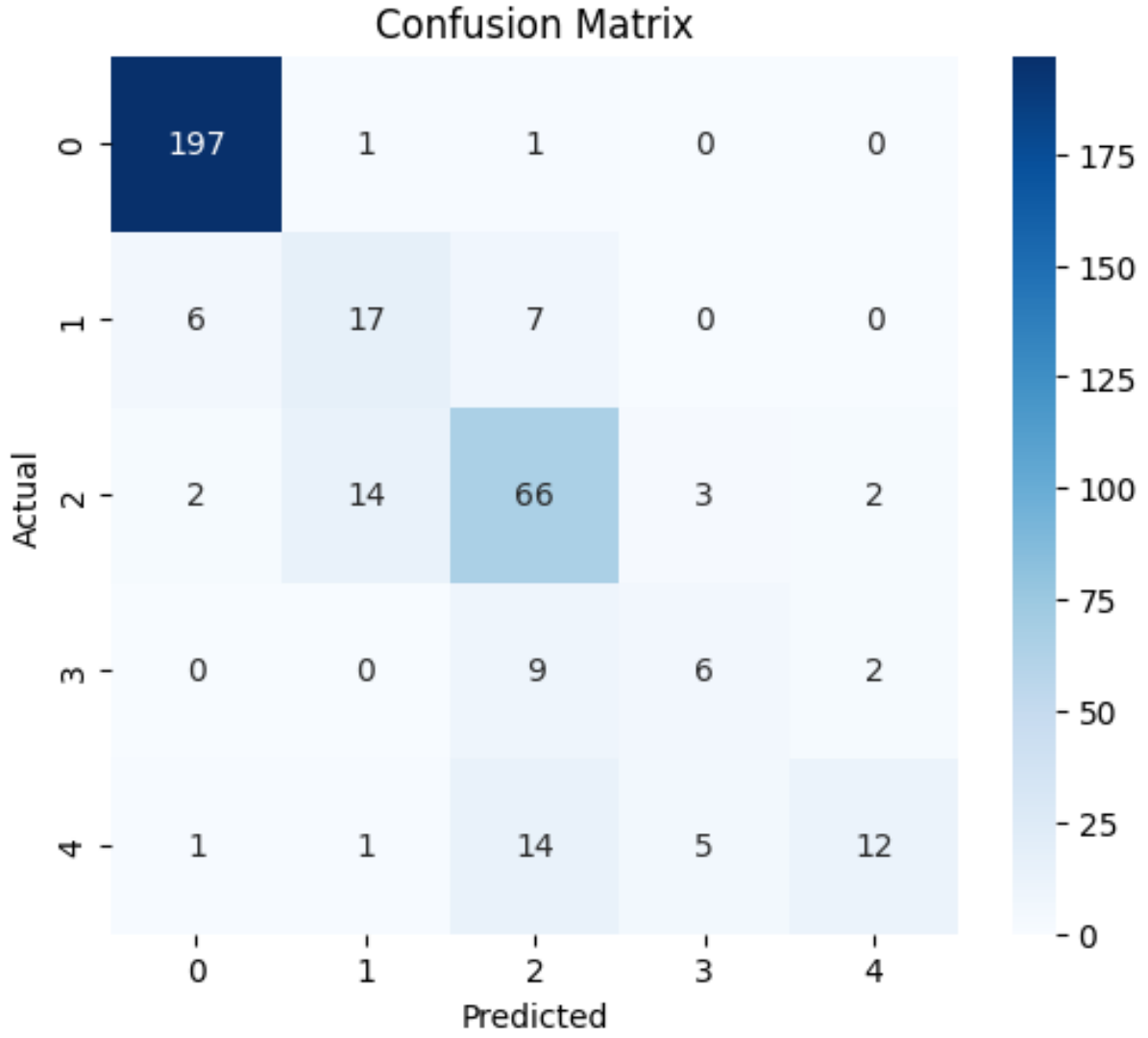


Figure 4: confusion matrix

	precision	recall	f1-score	support
0	0.9563	0.9899	0.9728	199
1	0.5152	0.5667	0.5397	30
2	0.6804	0.7586	0.7174	87
3	0.4286	0.3529	0.3871	17
4	0.7500	0.3636	0.4898	33
accuracy			0.8142	366
macro avg	0.6661	0.6064	0.6214	366
weighted avg	0.8115	0.8142	0.8059	366

Figure 5: precision recall f1score

5.7 Experiment 7: Automatic Hyperparameter Optimization

5.7.1 Baseline Model Configuration

The initial model was an ensemble of `efficientnet_b0` and `resnet18` that utilized Mixup data augmentation. The manually configured hyperparameters were as follows:

- Learning Rate: $1e{-}4$
- Optimizer: Adam
- Epochs: 20
- Dropout Rate: 0.5
- Mixup Alpha: 1.0

This baseline configuration achieved a validation accuracy of **86%**.

5.7.2 Optimization Strategy

To systematically improve upon the baseline, an automated hyperparameter tuning process was implemented with the following methodology:

- **Framework:** Optuna, a modern optimization library.
- **Search Strategy:** Bayesian Optimization. This intelligent search strategy uses results from previous trials to inform which hyperparameter combinations to test next. It was chosen for its efficiency, as it can find optimal parameters in fewer trials, which is ideal for computationally intensive models.
- **Efficiency Technique:** Pruning. This technique further accelerates the process by automatically stopping poorly performing trials early, preventing wasted computation.

5.7.3 Results and Analysis

Each trial was run for 10 epochs. However, the process was interrupted after completing only 10 trials.

The best-performing trial from this limited search yielded a validation accuracy of **84.4%** with hyperparameters were as follows:

- Learning Rate: $8.55427684640748e-05$
- Optimizer: RMSProp
- Epochs: 10
- Dropout Rate: 0.32740876046930245
- Mixup Alpha: 0.9551575924873654

A follow-up experiment was conducted to see if this model had further potential by retraining it from scratch with the same parameters but for 20 epochs. The key observation was that the accuracy did not improve beyond 84%, indicating the model had already reached its learning plateau and was limited by that specific set of hyperparameters.

5.7.4 Conclusion

The primary conclusion is that the 10-trial run was insufficient to explore the search space and discover a parameter set superior to the strong 86% baseline.

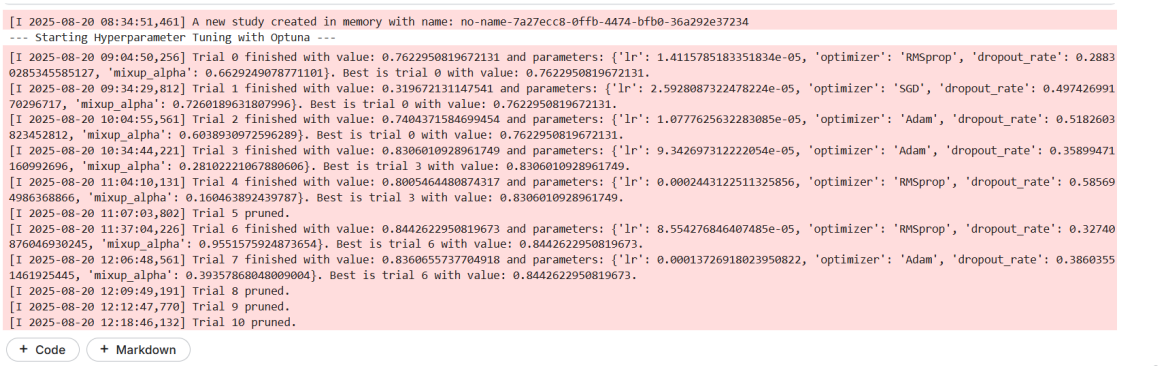


Figure 6: Automatic hyperparameter tuning using Optuna library

5.8 Experiment 8: Lesion-Aware Fusion with Attention MIL + CORN Loss

5.8.1 Objective

To improve diabetic retinopathy classification by combining **global retinal image features** and **lesion-guided local patches** within a Multiple Instance Learning (MIL) framework using **CORN loss** for ordinal regression.

5.8.2 Dataset & Preprocessing

- **Dataset**: APTOS 2019 Blindness Detection.
- **Input Strategy**:
  - Global View: 384×384 resized image.
  - Lesion-Guided Patches: 16 patches of size 224×224 extracted around bright lesions/hemorrhages using contour detection.
  - Random patches supplement lesion patches if insufficient lesions detected.
- **Transformations (Albumentations)**:
  - Global: Resize, CLAHE, flips, brightness/contrast, normalization.
  - Patches: RandomResizedCrop, CLAHE, flips, brightness/contrast, normalization.
  - Validation: CenterCrop + normalization only.

5.8.3 Model Architecture

- **Patch Encoder**: ResNet18 (pretrained).
- **MIL Attention**: Gated Attention pooling for lesion-aware patch aggregation.
- **Global Encoder**: EfficientNetV2-S (pretrained).
- **Fusion**: Concatenated embeddings → MLP.
- **Heads**:
  - Main head (ordinal regression using CORN loss).
  - Auxiliary head (patch-level supervision).

5.8.4 Training Configuration

- Optimizer: AdamW (learning rate = 2×10^{-4}, weight decay = 10^{-4}).
- Loss: CORN loss (main) + $\lambda\cdot$ auxiliary loss, with $\lambda = 0.2$.
- Batch size: 8, Epochs: 10.
- Device: CUDA (mixed precision with GradScaler).

5.8.5 Results

Training and Validation Losses (first 4 epochs):

Epoch	Train Main Loss	Train Aux Loss	Validation Loss
1	0.3070	0.4407	0.2876
2	0.2502	0.3861	0.2592
3	0.2294	0.3635	0.2392
4	0.2130	0.3632	0.2293

Table 3: Training and validation losses across epochs.

- Validation loss decreased consistently, showing that the model generalized well.
- CORN loss captured ordinal relationships between DR stages, reducing confusion between adjacent classes.
- Auxiliary loss converged steadily, indicating meaningful lesion feature learning.

5.8.6 Conclusion

The Lesion-Aware Fusion MIL model with CORN loss effectively reduced validation loss and improved ordinal consistency in DR classification.

- Attention-based patch aggregation highlighted lesion areas, leading to better interpretability.
- Combining global context with local lesion patches improved robustness compared to global-only models.
- Future work: explore transformer-based patch encoders (e.g., Swin Transformer, ViT), use focal loss or class-balanced loss to better handle minority classes, and visualize MIL attention maps for clinical interpretability.

6 Summary

Model	Techniques Applied	Class Balancing	Epochs	Validation Accuracy
Hybrid EfficientNet-B0 + ResNet18	None	No	4	81.00%
Hybrid EfficientNet-B0 + ResNet18	WeightedRandomSampler	Yes	4	82.24%
Hybrid EfficientNet-B0 + ResNet18	Weighted Cross-Entropy + Data Augmentation + Label Smoothing	No	4	82.79%
Hybrid EfficientNet-B0 + ResNet18	Same as above + Increased Epochs	No	30	83.06%
Hybrid EfficientNet-B0 + ResNet18	SMOTE + Mixup	Yes	30	84.00%
Hybrid EfficientNet-B0 + ResNet18	Mixup augmentation	Yes	20	86.00%
EfficientNet$_b$0 + $Resnet$18	Automatic Hyperparameter Optimization	Yes	20	86%
ResNet18+EfficientNet V2-S	Lesion-Aware Fusion with Attention MIL + CORN Loss	Yes	4	Val Loss-0.2293%
Hybrid EfficientNet-B4 + Swin Transformer	Standard Training	No	*Your Epochs*	82.00%

Table 4: Comparison of different model configurations, training techniques, and validation accuracies.

6.1 Performance Analysis

We conducted a series of experiments with different hybrid architectures, training strategies, and loss/augmentation techniques to evaluate their impact on classification performance. The baseline Hybrid EfficientNet-B0 + ResNet18 without any special sampling or augmentation achieved 81.00

Sampling and Loss Adjustments

Introducing a WeightedRandomSampler improved class balance and boosted accuracy to 82.24

Adding Weighted Cross-Entropy with Data Augmentation and Label Smoothing further improved the performance to 82.79

Increasing training epochs to 30 under the same setup gave a small but consistent improvement, achieving 83.06

Advanced Data Augmentation (SMOTE + Mixup)

The combination of SMOTE (synthetic oversampling) with Mixup yielded 84.00

Mixup Augmentation Alone

Pure Mixup augmentation for 20 epochs gave a significant jump to 86.00

Automatic Hyperparameter Optimization

Automated tuning of hyperparameters on the same hybrid model (EfficientNet-B0 + ResNet18) also reached 86.00

Advanced Fusion with Attention and Loss Functions

The ResNet18 + EfficientNet V2-S model with Lesion-Aware Fusion, Attention-based Multiple Instance Learning (MIL), and CORN loss achieved a validation loss of 0.2293, suggesting strong generalization ability and robust lesion-focused decision-making.

Transformer-based Hybrid

The EfficientNet-B4 + Swin Transformer hybrid achieved 82.00

6.1.1 Key Observations

1.Baseline Performance – The simple Hybrid EfficientNet-B0 + ResNet18 without sampling or augmentation achieved 81

2.Impact of Class Balancing – The use of WeightedRandomSampler improved accuracy from 81

3.Role of Loss Functions Regularization – Combining Weighted Cross-Entropy, Data Augmentation, and Label Smoothing pushed performance to 82.79

4.Training Duration – Increasing epochs from 4 $\rightarrow$ 30 only gave a marginal improvement (82.79

5.Effectiveness of Mixup – Mixup augmentation provided the biggest leap in performance, achieving 86

6.Synthetic Oversampling (SMOTE) + Mixup – Combining SMOTE and Mixup further boosted robustness to imbalance, reaching 84

7.Automated Hyperparameter Optimization – Auto-tuning matched Mixup's best performance (86

8.Advanced Fusion Strategies – The ResNet18 + EfficientNet V2-S with Lesion-Aware Fusion + Attention MIL + CORN loss demonstrated low validation loss (0.2293), suggesting that attention-based lesion-focused modeling improves reliability in medical image tasks.

9.Transformer Hybrids – The EfficientNet-B4 + Swin Transformer hybrid achieved 82

6.2 Conclusion and Final result

The baseline Hybrid EfficientNet-B0 + ResNet18 achieved 81

Mixup augmentation was the single most effective strategy, boosting performance to 86

Automatic hyperparameter optimization matched this peak performance, highlighting the importance of systematic tuning.

Advanced hybrid models like EfficientNet V2-S + ResNet18 with lesion-aware fusion and attention MIL demonstrated strong generalization with a low validation loss, showing promise for medical image tasks where lesion localization is critical.

The Transformer-based hybrid (EfficientNet-B4 + Swin) showed competitive accuracy (82

Overall, Mixup augmentation and automated hyperparameter tuning gave the best accuracy (86

6.3 Contribution of each team member:

By Sadiya : Experiment 1 ,2, 8

By Sanjeevni Kumari : Experiment 6 and 5

By Divya Arora: Experiment 3,4,7

Thank You!